PERSONAL FINANCE ADVISOR BOT

Project Documentation

AI-Powered Financial Management System

Name : Prithviraj Dhananjay Faratkhane

Project Repository:
github.com/prithviraj-0708/Personal-Finance-Advisor-Bot

Prepared for academic project documentation

Document Overview

This document describes the project's purpose, features, architecture, technology stack, setup, operation, testing, security considerations, limitations, and possible future improvements. It is based on the project's repository README and application entry point. Verify implementation details against the current code before final submission.

Table of Contents

1. 1. Abstract
2. 2. Introduction
3. 3. Problem Statement
4. 4. Aim and Objectives
5. 5. Scope of the Project
6. 6. Intended Users
7. 7. Main Features
8. 8. Technology Stack
9. 9. System Architecture
10. 10. Project Structure
11. 11. Database Overview
12. 12. Installation and Execution

13. 13. Environment Configuration
14. 14. How to Use the Application
15. 15. Testing
16. 16. Security and Privacy Considerations
17. 17. Limitations
18. 18. Future Enhancements
19. 19. Conclusion
20. 20. References

1. Abstract

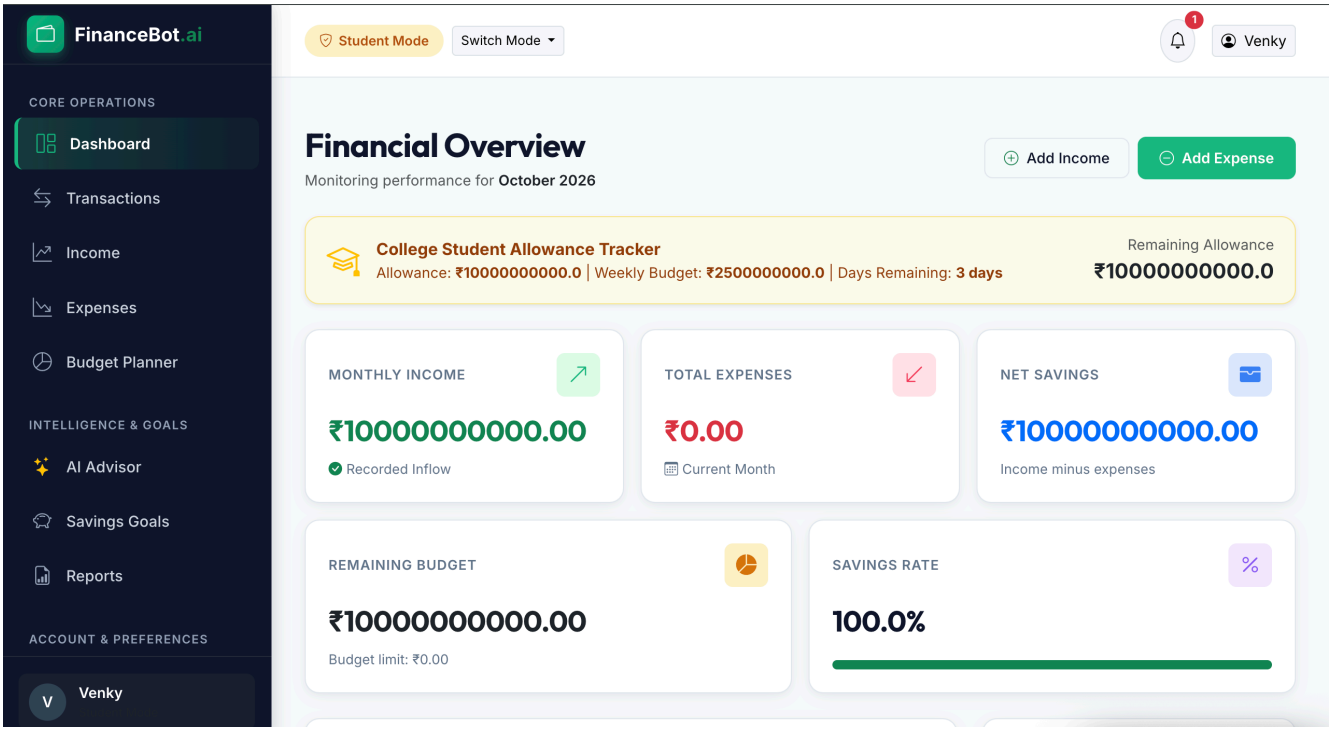
This project, Personal Finance Advisor Bot, is a web-based financial management application that helps users record income and expenses, plan budgets, track savings goals, review financial summaries, and receive personalized financial guidance. It combines a Flask backend, an SQLite database, a responsive browser interface, financial calculation services, and optional Google Gemini AI integration.

The application is designed for students, freelancers, households, and general users. When a Gemini API key is unavailable or the service cannot be reached, the application can use a built-in rule-based fallback engine for supported advisory responses. The system is intended for financial awareness and planning support, not as a replacement for a qualified financial professional.

2. Introduction

Managing personal finances can be difficult when income, expenses, budgets, and savings goals are recorded separately or not reviewed regularly. Users may struggle to identify unnecessary spending, understand their monthly cash flow, or determine how much to save toward a target.

The Personal Finance Advisor Bot brings these tasks into one application. Users can maintain financial records, view summaries and charts, create budgets, set savings goals, and ask questions through an AI-assisted advisor interface.



3. Problem Statement

Many individuals do not have a simple, centralized way to track day-to-day financial activity and turn those records into understandable actions. Manual calculations and scattered spreadsheets can make it difficult to identify spending patterns, maintain category limits, and monitor progress toward savings goals.

The project addresses this problem by providing an integrated, browser-based system for recording transactions, organizing spending, monitoring budgets, tracking goals, and generating understandable financial summaries.

4. Aim and Objectives

Aim: To develop a user-friendly personal finance web application that supports financial tracking, budgeting, savings planning, reporting, and AI-assisted guidance.

Objectives:

- Provide secure account registration and login.
- Record and manage income and expense entries.
- Categorize transactions and support search, filtering, and sorting.
- Generate budgets based on income, spending patterns, and the selected financial mode.
- Display budget utilization warnings and over-budget status.
- Let users define savings goals and record contributions.
- Present financial summaries and charts for easier understanding.
- Export financial records to CSV and provide a printable monthly statement.
- Offer AI-assisted responses using Google Gemini when configured, with rule-based fallback behavior for supported queries.
- Support demo data for presentations and academic evaluation.

5. Scope of the Project

The application covers personal income and expense management, category-based budgets, savings goals, dashboard summaries, transaction filtering, visual reports, CSV exports, and an AI advisor interface. It includes specialized modes for college students, freelancers, households, and standard personal budgeting.

The current project is a personal finance planning and tracking tool. It does not connect directly to bank accounts, execute payments, guarantee investment returns, or replace professional tax, legal, or investment advice.

6. Intended Users

- College students managing allowances, transport, food, books, and entertainment spending.
- Freelancers managing irregular project payments and work-related expenses.
- Households tracking shared groceries, rent, utilities, and savings targets.
- Individuals who want a straightforward record of income, expenses, budgets, and savings.

7. Main Features

7.1 User Accounts: Registration, login, session management, and user-specific financial records.

7.2 Income Management: Record income amounts, sources, dates, notes, and recurring monthly income where supported.

7.3 Expense Management: Record spending by category and payment method, with notes and relevant work-expense tags.

7.4 Transaction Ledger: Search and filter income and expense records by category and date range, and sort by date or amount.

7.5 Dashboard and Summaries: Show key financial metrics, daily spending, rolling seven-day totals, monthly cash flow, and alerts.

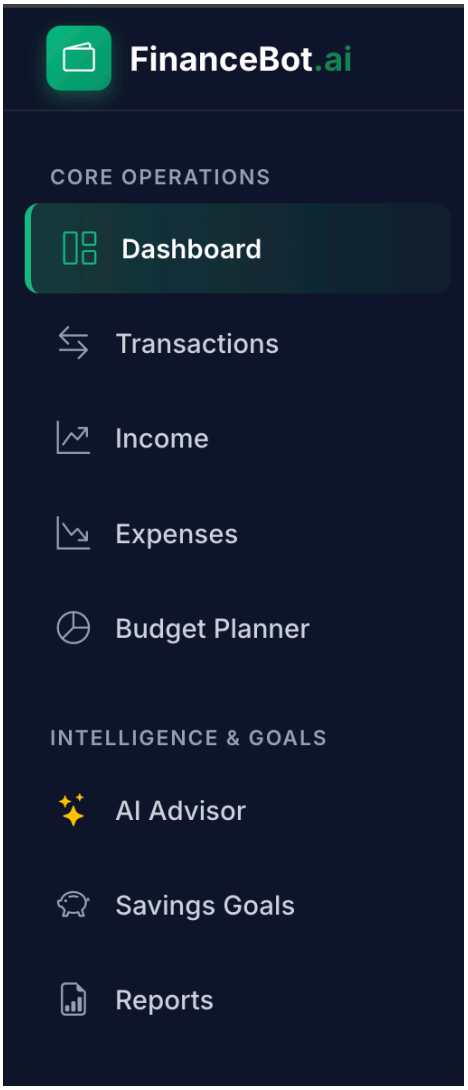
7.6 Budget Planner: Generate category limits using income, previous spending, and the selected mode. Display progress statuses: on track below 80%, warning from 80% to below 100%, and over budget at or above 100%.

7.7 AI Financial Advisor: Answer supported questions about the user's financial records and provide spending observations and savings suggestions. Gemini integration is optional; a rule-based fallback is available for supported behavior.

7.8 Savings Goals: Create target amounts and dates, record contributions, and calculate the monthly savings pace needed to reach goals.

7.9 Reports and Exports: Display charts for income versus expenses, category spending, savings trends, and budget utilization. Export supported records as CSV and print a monthly statement.

7.10 Demo Data: Populate sample transactions, budgets, and goals to make demonstrations and testing easier.



8. Technology Stack

Backend: Python 3.9 or later, Flask, Werkzeug, python-dotenv.

Database and ORM: SQLite and Flask-SQLAlchemy.

Authentication and security: Flask-Login, password hashing through Werkzeug, Flask-WTF, CSRF protection, and email-validator.

AI integration: Google GenAI Python SDK, configured for Gemini 2.5 Flash in the project README.

Frontend: HTML5, CSS3, JavaScript, Jinja2 templates, Bootstrap 5, Bootstrap Icons, and Chart.js.

Testing: pytest and test fixtures using an in-memory database.

9. System Architecture

The application follows a modular Flask architecture. A browser sends requests to Flask routes. Route blueprints validate inputs and coordinate the relevant service logic. Services perform calculations, analytics, budget planning, AI interaction, and report generation. SQLAlchemy models represent stored financial records in SQLite. Jinja2 templates render pages, while JavaScript and Chart.js provide interactive behavior and visualizations.

Typical request flow: User Interface → Flask Route / Blueprint → Service Layer → SQLAlchemy Models → SQLite Database → Response rendered in the browser.

AI request flow: AI Advisor page → advisor route → Gemini service → Gemini API when configured and available; otherwise, the supported rule-based fallback path is used.

10. Project Structure

app.py — application factory, Flask setup, login manager, CSRF protection, template filters, route registration, error handlers, and table creation.

config.py — application and environment configuration.

models.py — database models for users and financial entities.

forms.py — form definitions and validation.

routes/ — modular route handlers for authentication, dashboard, income, expenses, budgets, savings, reports, and AI advisor.

services/ — analytics_service.py, budget_service.py, gemini_service.py, and report_service.py for business logic.

templates/ — Jinja2 HTML pages for the landing page, authentication, dashboard, transactions, budgets, advisor, savings, reports, settings, and error pages.

static/css/ — application styling.

static/js/ — general interface behavior, charts, and AI advisor messaging.



instance/finance.db — SQLite database file generated by the application at runtime.

tests/ — automated tests for authentication, income, expenses, budgets, savings, and AI fallback behavior.

requirements.txt — Python package dependencies; .env.example — environment-variable template.

11. Database Overview

The application uses SQLite through Flask-SQLAlchemy. The database is created automatically when the application starts, using the models defined in the project. The README identifies entities including User, Income, Expense, Budget, SavingsGoal, and Notification.

User records support account ownership and authentication. Income and expense records store financial transactions. Budget records represent category limits and utilization. Savings goals store targets and progress, while notifications support alerts shown in the interface.

Exact column names, relationships, and constraints should be confirmed against the current models.py file before preparing a database schema diagram or a formal data dictionary.

12. Installation and Execution

Prerequisites: Python 3.9 or later, pip, a terminal, and a browser. VS Code is recommended but not mandatory.

Step 1 — Clone the repository: git clone https://github.com/prithviraj-0708/Personal-Finance-Advisor-Bot.git

Step 2 — Enter the project directory: cd Personal-Finance-Advisor-Bot

Step 3 — Create a virtual environment: python3 -m venv venv

Step 4 — Activate it on macOS/Linux: source venv/bin/activate. On Windows Command Prompt: venv\\Scripts\\activate.bat. On Windows PowerShell: venv\\Scripts\\Activate.ps1.

Step 5 — Install packages: pip install -r requirements.txt

Step 6 — Create the environment file: cp .env.example .env (on Windows, copy the file using File Explorer or the appropriate copy command).

Step 7 — Configure a strong FLASK_SECRET_KEY. Optionally configure GEMINI_API_KEY and GEMINI_MODEL. Set DATABASE_URL and DEFAULT_CURRENCY if needed, following the project's configuration.

Step 8 — Start the application: python app.py

Step 9 — Open http://127.0.0.1:5000 in a browser. The database tables are created automatically at startup according to the application code.

13. Environment Configuration

FLASK_SECRET_KEY — secret used by Flask for session and security-related signing; use a strong private value.

GEMINI_API_KEY — optional Google Gemini API key. Leave unset to use the fallback behavior where supported.

GEMINI_MODEL — model name used by the AI integration; the project README specifies gemini-2.5-flash.

DATABASE_URL — database connection string; the README gives sqlite:///finance.db as an example, while the app may use the instance directory based on its configuration.

DEFAULT_CURRENCY — currency symbol displayed by the application; the README uses ₹.

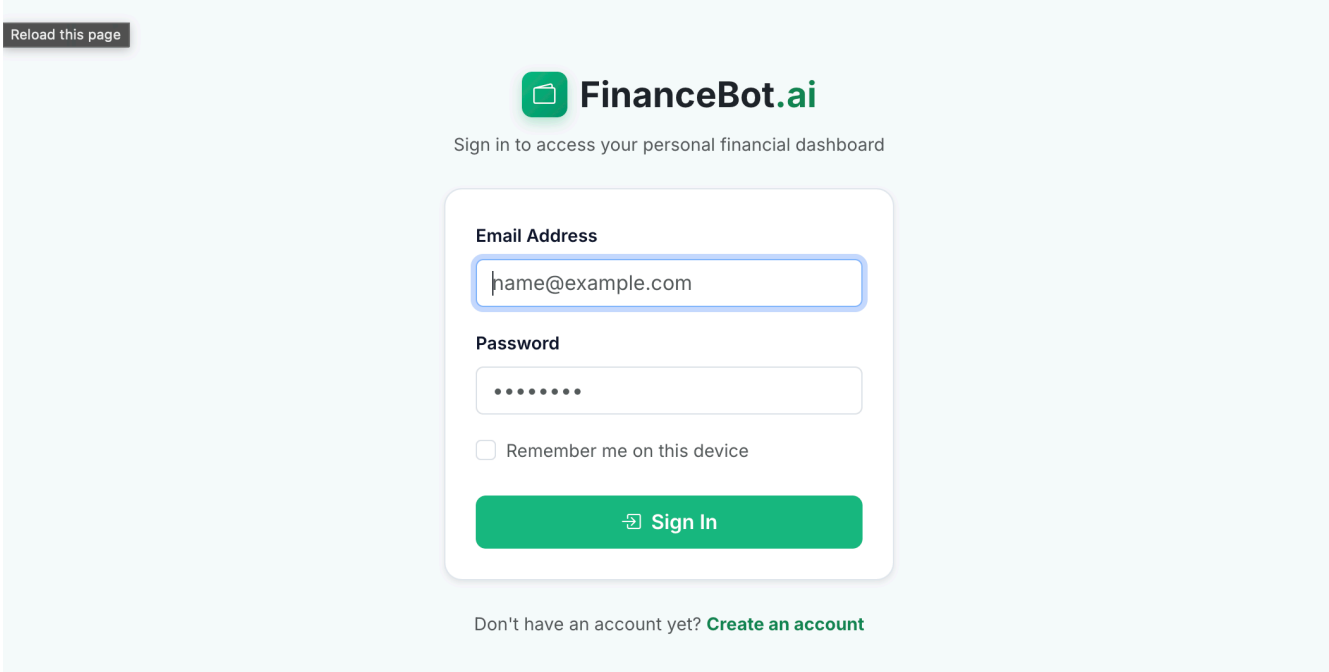
Never commit real API keys, production secrets, or personal financial data to a public repository. Keep .env private.

14. How to Use the Application

21. Open the application and create an account.
22. Sign in and review the dashboard.
23. Add income entries with their source, amount, and date.
24. Add expenses with a category, payment method, and notes if needed.
25. Open the transaction ledger to search, filter, and review records.
26. Use the budget planner to generate or manage category limits and review warning indicators.
27. Create savings goals and record contributions as progress is made.
28. Open the AI Advisor and try questions suggested by the interface. Verify any important totals against the ledger.

29. Open Reports to review charts, export supported records to CSV, or print the monthly statement.

30. For a presentation, use the demo-data option in Settings if available in the running version.



15. Testing

The project README describes a pytest-based test suite with 16 tests. The listed test areas include authentication and session isolation, income and expense operations, budget limits and calculations, student-mode behavior, savings goals, and AI fallback responses.

Run tests from the project root after installing dependencies: pytest -v

Testing should be repeated after code changes. Test results should be recorded from the actual local run; this document does not claim that the test suite was executed as part of preparing this report.

16. Security and Privacy Considerations

Use a strong secret key and keep it out of source control. CSRF protection is initialized in the application, and authentication is managed with Flask-Login. Password storage should use the project's configured secure password-hashing implementation.

Each financial record should be checked for correct user ownership before reading, editing, exporting, or deleting it. Validate user inputs and avoid exposing exception details in production. Use HTTPS when deployed, keep dependencies updated, and avoid using real bank statements in public demos.

AI-generated suggestions may be incomplete or inaccurate. Users should verify calculations and avoid sharing account credentials, full card numbers, passwords, or other sensitive information with the chatbot. The tool should not be presented as regulated financial advice.

17. Limitations

AI responses depend on the availability, configuration, and behavior of the selected model; fallback responses may be less flexible than Gemini responses.

The accuracy of budgets and recommendations depends on the quality and completeness of user-entered data.

The application is not described as having direct bank synchronization or automatic payment capabilities.

Deployment-specific persistence, backups, HTTPS, monitoring, and multi-user production hardening must be checked separately before real-world use.

Financial advice generated by software is informational and should be independently reviewed.

18. Future Enhancements

- Add optional bank statement import with explicit user review before saving transactions.
- Improve recurring transaction detection and configurable reminders.
- Add downloadable PDF reports and richer month-to-month comparisons.
- Add database backup and restore tools.
- Strengthen deployment configuration, audit logging, rate limiting, and privacy controls.
- Add multilingual support and accessible interface improvements.
- Expand automated tests and add tests for edge cases and authorization boundaries.
- Provide clearer explanations and sources for AI-generated financial guidance.

19. Conclusion

The Personal Finance Advisor Bot combines transaction management, budgeting, savings goals, reporting, and AI-assisted guidance in a single web application. Its modular Flask structure separates routes, business logic, data models, and interface components, making the project suitable for learning and further development.

The system can help users understand their cash flow and build better budgeting habits. Before production use, it should be tested in its target deployment environment, and all financial recommendations should be treated as informational rather than guaranteed or professional advice.

20. References

Project repository: https://github.com/prithviraj-0708/Personal-Finance-Advisor-Bot

Flask documentation: https://flask.palletsprojects.com/

Flask-SQLAlchemy documentation: https://flask-sqlalchemy.palletsprojects.com/

Flask-Login documentation: https://flask-login.readthedocs.io/

Google AI for Developers: https://ai.google.dev/

Chart.js documentation: https://www.chartjs.org/docs/